Insertion

DSA LAB-10
9.3.26
NAME: Ninada S A
USN: 1RUA25BCA0075

1.Insertion at the beginning of a doubly circular link-list:

Code-

```c
#include <stdio.h>
#include <stdlib.h>
struct DNode {
int data;
struct DNode* next;
struct DNode* prev;
};
void display(struct DNode* head) {
if(head == NULL) return;
struct DNode* temp = head;
printf("%d <-> ", temp->data);
temp = temp->next;
while(temp != head) {
printf("%d <-> ", temp->data);
temp = temp->next;
}
printf("(Back to Head)\n");
}
void insertBeg(struct DNode** head, int val) {
struct DNode* newNode = (struct DNode*)malloc(sizeof(struct DNode));
newNode->data = val;
struct DNode* last = (*head)->prev;
newNode->next = *head;
newNode->prev = last;
last->next = newNode;
(*head)->prev = newNode;
*head = newNode;
}
int main() {
struct DNode* n1 = (struct DNode*)malloc(sizeof(struct DNode));
struct DNode* n2 = (struct DNode*)malloc(sizeof(struct DNode));
struct DNode* n3 = (struct DNode*)malloc(sizeof(struct DNode));
struct DNode* n4 = (struct DNode*)malloc(sizeof(struct DNode));
n1->data=10; n2->data=20; n3->data=30; n4->data=40;
n1->next=n2; n2->next=n3; n3->next=n4; n4->next=n1;
n1->prev=n4; n2->prev=n1; n3->prev=n2; n4->prev=n3;
struct DNode* head = n1;
struct DNode* tail = n4;
insertBeg(&head, 5);
display(head);
return 0;
}
```

Output-

```
Output
5 <-> 10 <-> 20 <-> 30 <-> 40 <-> (Back to Head)

=== Code Execution Successful ===
```

2.Insertion at the end of a doubly circular link-list:

Code-

```c
#include <stdio.h>
#include <stdlib.h>
struct DNode {
int data;
struct DNode* next;
struct DNode* prev;
};
void display(struct DNode* head) {
if(head == NULL) return;
struct DNode* temp = head;
printf("%d <-> ", temp->data);
temp = temp->next;
while(temp != head) {
printf("%d <-> ", temp->data);
temp = temp->next;
}
printf("(Back to Head)\n");
}
void insertEnd(struct DNode** head, int val) {
struct DNode* newNode = (struct DNode*)malloc(sizeof(struct DNode));
newNode->data = val;
struct DNode* last = (*head)->prev;
newNode->next = *head;
newNode->prev = last;
last->next = newNode;
(*head)->prev = newNode;
}
int main() {
struct DNode* n1 = (struct DNode*)malloc(sizeof(struct DNode));
struct DNode* n2 = (struct DNode*)malloc(sizeof(struct DNode));
struct DNode* n3 = (struct DNode*)malloc(sizeof(struct DNode));
struct DNode* n4 = (struct DNode*)malloc(sizeof(struct DNode));
n1->data=10; n2->data=20; n3->data=30; n4->data=40;
n1->next=n2; n2->next=n3; n3->next=n4; n4->next=n1;
n1->prev=n4; n2->prev=n1; n3->prev=n2; n4->prev=n3;
struct DNode* head = n1;
struct DNode* tail = n4;
insertEnd(&head, 50);
display(head);
return 0;
```

Output-

```
5 <-> 10 <-> 20 <-> 30 <-> 40 <-> (Back to Head)
```

3.Insertion at a given position of a doubly circular link-list:

Code-

```c
#include <stdio.h>
#include <stdlib.h>
struct DNode {
int data;
struct DNode* next;
struct DNode* prev;
};
void display(struct DNode* head) {
if(head == NULL) return;
struct DNode* temp = head;
printf("%d <-> ", temp->data);
temp = temp->next;
while(temp != head) {
printf("%d <-> ", temp->data);
temp = temp->next;
}
printf("(Back to Head)\n");
}
void insertPos(struct DNode** head, int val, int pos) {
struct DNode* newNode = (struct DNode*)malloc(sizeof(struct DNode));
newNode->data = val;
struct DNode* temp = *head;
for(int i=1; i<pos-1; i++) temp = temp->next;
newNode->next = temp->next;
newNode->prev = temp;
temp->next->prev = newNode;
temp->next = newNode;
}
int main() {
struct DNode* n1 = (struct DNode*)malloc(sizeof(struct DNode));
struct DNode* n2 = (struct DNode*)malloc(sizeof(struct DNode));
struct DNode* n3 = (struct DNode*)malloc(sizeof(struct DNode));
struct DNode* n4 = (struct DNode*)malloc(sizeof(struct DNode));
n1->data=10; n2->data=20; n3->data=30; n4->data=40;
n1->next=n2; n2->next=n3; n3->next=n4; n4->next=n1;
n1->prev=n4; n2->prev=n1; n3->prev=n2; n4->prev=n3;
struct DNode* head = n1;
struct DNode* tail = n4;
insertPos(&head, 25, 3);
display(head);
return 0;
}
```

Output-

```
10 <-> 20 <-> 25 <-> 30 <-> 40 <-> (Back to Head)
```

Deletion

DSA LAB-10
9.3.26
NAME: Ninada S A
USN: 1RUA25BCA0075

1.Deletion at the beginning of a doubly circular link-list:

Code-

```c
#include <stdio.h>
#include <stdlib.h>
struct DNode {
int data;
struct DNode* next;
struct DNode* prev;
};
void display(struct DNode* head) {
if(head == NULL) return;
struct DNode* temp = head;
printf("%d <-> ", temp->data);
temp = temp->next;
while(temp != head) {
printf("%d <-> ", temp->data);
temp = temp->next;
}
printf("(Back to Head)\n");
}
void deleteBeg(struct DNode** head) {
struct DNode* temp = *head;
struct DNode* last = (*head)->prev;
last->next = temp->next;
temp->next->prev = last;
*head = temp->next;
free(temp);
}
int main() {
struct DNode* n1 = (struct DNode*)malloc(sizeof(struct DNode));
struct DNode* n2 = (struct DNode*)malloc(sizeof(struct DNode));
struct DNode* n3 = (struct DNode*)malloc(sizeof(struct DNode));
struct DNode* n4 = (struct DNode*)malloc(sizeof(struct DNode));
n1->data=10; n2->data=20; n3->data=30; n4->data=40;
n1->next=n2; n2->next=n3; n3->next=n4; n4->next=n1;
n1->prev=n4; n2->prev=n1; n3->prev=n2; n4->prev=n3;
struct DNode* head = n1;
struct DNode* tail = n4;
deleteBeg(&head);
display(head);
return 0;
}
```

Output-

```
Output
20 <-> 30 <-> 40 <-> (Back to Head)
```

2.Deletion at the end of a doubly circular link-list:

Code-

```c
#include <stdio.h>
#include <stdlib.h>
struct DNode {
int data;
struct DNode* next;
struct DNode* prev;
};
void display(struct DNode* head) {
if(head == NULL) return;
struct DNode* temp = head;
printf("%d <-> ", temp->data);
temp = temp->next;
while(temp != head) {
printf("%d <-> ", temp->data);
temp = temp->next;
}
printf("(Back to Head)\n");
}
void deleteEnd(struct DNode** head) {
struct DNode* last = (*head)->prev;
struct DNode* secondLast = last->prev;
secondLast->next = *head;
(*head)->prev = secondLast;
free(last);
}
int main() {
struct DNode* n1 = (struct DNode*)malloc(sizeof(struct DNode));
struct DNode* n2 = (struct DNode*)malloc(sizeof(struct DNode));
struct DNode* n3 = (struct DNode*)malloc(sizeof(struct DNode));
struct DNode* n4 = (struct DNode*)malloc(sizeof(struct DNode));
n1->data=10; n2->data=20; n3->data=30; n4->data=40;
n1->next=n2; n2->next=n3; n3->next=n4; n4->next=n1;
n1->prev=n4; n2->prev=n1; n3->prev=n2; n4->prev=n3;
struct DNode* head = n1;
struct DNode* tail = n4;
deleteEnd(&head);
display(head);
return 0;
}
```

Output-

```
10 <-> 20 <-> 30 <-> (Back to Head)
```

3.Deletion at a given position of a doubly circular link-list:

Code-

```
#include <stdio.h>
#include <stdlib.h>
struct DNode {
int data;
struct DNode* next;
struct DNode* prev;
};
void display(struct DNode* head) {
if(head == NULL) return;
struct DNode* temp = head;
printf("%d <-> ", temp->data);
temp = temp->next;
while(temp != head) {
printf("%d <-> ", temp->data);
temp = temp->next;
}
printf("(Back to Head)\n");
}
void deletePos(struct DNode** head, int pos) {
struct DNode* temp = *head;
for(int i=1; i<pos; i++) temp = temp->next;
temp->prev->next = temp->next;
temp->next->prev = temp->prev;
free(temp);
}
int main() {
struct DNode* n1 = (struct DNode*)malloc(sizeof(struct DNode));
struct DNode* n2 = (struct DNode*)malloc(sizeof(struct DNode));
struct DNode* n3 = (struct DNode*)malloc(sizeof(struct DNode));
struct DNode* n4 = (struct DNode*)malloc(sizeof(struct DNode));
n1->data=10; n2->data=20; n3->data=30; n4->data=40;
n1->next=n2; n2->next=n3; n3->next=n4; n4->next=n1;
n1->prev=n4; n2->prev=n1; n3->prev=n2; n4->prev=n3;
struct DNode* head = n1;
struct DNode* tail = n4;
deletePos(&head, 3);
display(head);
return 0;
}
```

Output-

```
10 <-> 20 <-> 40 <-> (Back to Head)
```